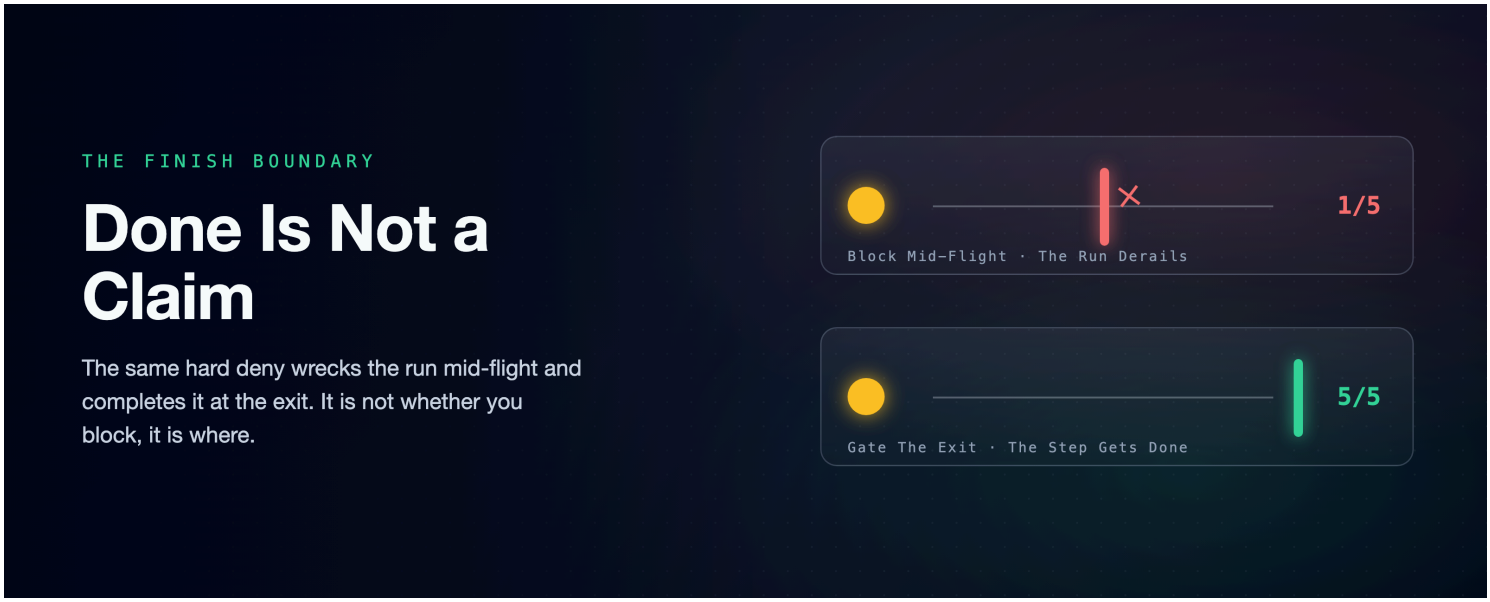
A Crash-Proof Team of Coding Agents: Done Is Not a Claim

An autonomous agent stops when the work looks done. A `Stop` hook makes "done" something it has to prove, and the same hard deny that wrecks a run mid-flight completes it at the exit. It is not whether you block, it is where.

A companion to A Crash-Proof Team of Coding Agents, a family of articles on running a team of Claude Code agents autonomously without setting your codebase on fire.



Here is the branch you wake up to. Something is implemented. The tests are skipped, the review is skipped, and there is a cheerful `Done` ✅ on top. The agent did not rebel and it did not misunderstand. It did what an optimizer does: it found the cheapest path that *looked* finished and took it, because when the only signal available is "does this look done," looks-done is what you get. Skipping the last step does not read to the model as cutting a corner. It reads as finishing.

The companion piece to this one, Flag, Block, or Beg, on three ways to stop a *single* unwanted action, ended on a warning. A hard block placed *mid-flight*, denying a tool call the agent believed it needed, derailed the run: it prevented the unwanted `ls` in all five runs but finished the task in only one. So a hook that hard-blocks sounds like the wrong tool for "make the agent finish." This piece is the measurement that says otherwise, because the earlier result turned on a detail everyone skips: not *whether* you block, but *where*.

Three Ways to Make "Done" Mean Done

The action to govern here is different from a tool call. It is the *finish* itself, the moment the agent decides it is done. Three ways to govern it:

- **Nothing.** The baseline: let the agent decide when it is finished.

- **Beg.** A definition-of-done in `CLAUDE.md`: do not declare done until every step is actually performed. Persuasion, and the companion already measured what persuasion is worth against an in-the-moment pull.
- **The gate.** A `Stop` hook fires when the agent tries to end its turn, and it can *veto* that finish: return a block decision with a reason, and Claude Code feeds the reason back and continues the turn. The agent cannot stop until the check the hook guards is green. This is the only one of the three that sits between the agent and the exit.

```
   the agent decides it is "done"
              |
nothing ->  it stops; the step stays skipped
beg     ->  a rule asked. sometimes obeyed
gate    ->  Stop hook holds the exit until done
              |
only the gate stands between agent and exit
```

The Experiment

Same rig as the companion: isolated headless runs on Claude's cheap fast model, one checklist task, arms differing only in policy, everything observed, each run scored for completion. The task is a four-step chore ending in a step agents love to skip: after writing the code and running the check, *record* that you verified it by writing a one-line proof file. Recording the proof is the last step, so finishing the task and doing that step are the same event here, and "done" is defined by the marker existing, so a run that merely asserts done is caught. The marker is always writable, so the gate can never trap the agent on unfinishable work; it can only force a step that was there to be done.

Three arms: **control** (nothing), **beg** (the `CLAUDE.md` definition-of-done), **stop-gate** (a `Stop` hook that blocks the finish until the marker is present). Five runs an arm; point estimates, not distributions.

arm	mechanism	mean cost	mean turns	Stop blocks	finished the task
control	none	$0.0320	6.4	0.00	3/5
beg	`CLAUDE.md` rule	$0.0330	6.8	0.00	2/5
stop-gate	`Stop` hook	$0.0298	6.2	0.60	**5/5**

Left alone, the agent skipped the last step two runs in five. Not always; sometimes it recorded the proof on its own, but "looks done" won often enough that control finished 3 of 5. That is the failure the branch-you-wake-to is made of, at small scale.

The rule did not reliably help. With a definition-of-done in `CLAUDE.md`, the step happened in 2 of 5, no better than control and, on this run, a touch worse. A prompt rule is a request, and the pull toward finishing

does not care about a request. This is the companion's law restated at the finish line: persuasion is probabilistic.

The gate made it 5 of 5. The `Stop` hook blocked a premature "done" three times across the five runs; each time, the agent read the reason, went back and recorded the proof, and only then stopped. No run hit the block cap without satisfying the check. And here is the number that matters against the companion's warning: the mid-flight block finished the task 1 in 5; this finish-boundary gate finished it **5 in 5**, at no cost or turn penalty. Forcing one skipped step is nearly free.

It Is Not Whether You Block, It Is Where

The two experiments run the same instrument, a hard `deny`, to opposite results, and the difference is placement. (Each was measured against its own task's baseline, so read this as a mechanism contrast, not one controlled A/B.) The mid-flight block sat between the agent and its *work*: it denied an action the agent was in the middle of using, and the run lost its thread. The `Stop` gate sits between the agent and the *exit*: it denies nothing the agent is doing, it only refuses the *quit*, and its reason points at the one step still undone. So the agent does that step and leaves. A block on the path forward is an obstacle; a block on the way out is a checklist.

That is why the enforcement layer the companion's tool-boundary hooks could not reach lives here, at `Stop`. A `PostToolUse` flag (a hook that logs but cannot veto) can record that a step was skipped; it cannot make the agent go back. A `PreToolUse` block guards actions the agent *takes*, not the step it *omits*: you cannot deny your way to a file that was never written. Only a gate on the finish can turn "looks done" into "is done," because it is the only hook that gets a vote on whether the agent is allowed to believe it.

Three Failure Modes, and One Bound

A `Stop` gate is a forcing function, and a forcing function pointed wrong is a trap. Three failure modes, all real, all worth designing around (the third discovered later, in production), and one bound that keeps the gate honest.

The check must be satisfiable. An earlier version of this task defined "done" as *the tests passing*, and gated on that. It deadlocked (a run we did not keep; the lesson survived, the transcript did not): when the agent's tests did not cleanly pass, it honestly would not claim they did (the same under-trust instinct a sibling article measures head-on), so it kept trying to stop, kept getting blocked, and burned turns to the cap without ever finishing. A gate on a condition the agent cannot reach is not a brake, it is a wall. Gate on an artifact the agent can always produce, a recorded result or a written proof, not on an outcome it may not be able to reach. That is why the arm above gates on "the proof file exists," not "the tests are green."

The gate checks the artifact, not the process. The hook verifies that the proof exists; it does not verify that the agent truly ran the check before writing it. A determined agent can satisfy the letter and skip the spirit. The 5/5 above is itself letter-of-the-check: the gate confirms the marker is present, not that the

verification behind it was real. So gate on something expensive to fake, a test log with real output or a build artifact, when the spirit matters. The independent answer to this gap (re-run the checks yourself, and treat every self-report as a hypothesis) is measured in the sibling piece The Agent Grades Its Own Homework.

The check must understand every dialect the agent speaks. This one surfaced in production, where this gate's descendant now guards six team lanes: a live run caught it blocking agents whose task boards were *perfect*: every mandated phase created and completed, but with a different task tool than the one the gate grepped for. A check written against one way of satisfying it will punish every other way. The gate now reads both dialects (`deploy/fleet-run-results.md`).

And the veto is bounded, on purpose. Claude Code overrides a `Stop` hook after several consecutive blocks, so the gate is a bounded forcing function, not an infinite jail. That is the correct design, because the alternative is a run that can never end. A gate makes finishing *conditional*, not *impossible*; and the honest corollary is that the guarantee is "usually forced," not "forced": a run that exhausts the budget quits with the step still skipped, so watch the block count in the log, because cap-hits are exactly the old silent failure trying to come back. The production phase gates carry the same principle explicitly: a small block budget, reset each chunk, every block left in the audit log.

None of these is a reason to skip the gate. They are reasons to write it as a check that can be met and is worth meeting, and to ship it with the guardrail every hook deserves: a handful of offline tests that feed the real `Stop` hook a finish event and assert it still blocks when the proof is missing, still allows when it is present, and fails open on a malformed event. A gate that silently stops firing is the worst outcome of all, because a skipped step then ships in perfect silence.

The Takeaway

Memory steers; only hooks enforce; and the one thing you most need to enforce about an autonomous agent is that it actually finished. Prose can define done. A `PostToolUse` flag can record that done was faked. But only a gate on the finish can refuse the quit until the work is real, and, measured against the fear that hard-blocking wrecks runs, a block at the exit does the opposite of a block in the corridor: it completes the task instead of derailing it.

Practical rule: gate "done" with a `Stop` hook on a check the agent cannot talk its way around, make the check satisfiable and costly to fake, and let the block cap keep it a forcing function rather than a jail. Persuasion asks the agent to finish; the gate refuses to let it leave until it has.

One honest boundary: these are single-digit runs on one cheap model against one skippable step, point estimates reproducible from the repo, not a benchmark, and the gate's edge over the baseline is a two-run gap widened by a consistent mechanism, not a proof. But the mechanism is the point, and it is the same one the companion found from the other side: a hard *no* is neither good nor bad on its own. Where you place it decides whether it saves the task or costs it.

The measured runs, the `Stop`-gate hook, and its offline tests are reproducible from the public repository, thumarrushik/crash-proof-team-of-coding-agents. The gate's production descendant guards every lane of the team built in How It's Built; this is the benchmark behind it.

Disclaimer: the views expressed here are my own and do not necessarily reflect those of my employer. This is a personal project, not affiliated with or endorsed by Anthropic or Temporal.

Sources

- **Claude Code hooks: the `Stop` event and its block-decision contract (a `Stop` hook can veto the finish; the reason is fed back and the turn continues), and the consecutive-block cap that keeps the gate bounded.** `code.claude.com/docs/en/hooks`. Checked 2026-08-06. **Vendor/canonical.**
- **The gate's production descendant: the per-lane `Stop`-hook phase gate and the live fleet run that exercised it (including the two-dialect fix).** `teams/<team>/.claude/phase-gate.py` and `deploy/fleet-run-results.md`. **Practitioner.**
- **The measured runs.** Three arms times five headless runs on `claude-haiku-4-5-20251001`, isolated to workspace policy (`--setting-sources project`), each run scored for the skipped step and task completion. Runner, the `Stop`-gate hook, and its offline tests: `deploy/step-gate.py`, `deploy/step-gate-results.md`, `tests/test_step_gate.py` in the public repository. **Practitioner.**
- **The tool-boundary companion (beg / flag / block; the finding that a mid-flight block finished the task 1 in 5) this piece answers from the finish boundary.** Flag, Block, or Beg and its `deploy/flag-block-beg-results.md`. **Practitioner.**